Task 1: Web App QA & Debug Report — Ronit Gulia

Target Application: Conduit Live Demo (https://demo.realworld.io)

Flows Tested: User Sign-up, Login, Create Article, Edit/Delete Article, Favourite Toggle, Global Feed Pagination, and Session Logout.

1. Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**Missing Idempotency & Button Debouncing on "Publish Article" Causes Duplicate Posts**	1. Log in to demo.realworld.io. 2. Go to **New Article** and fill in Title, Description, and Body. 3. Throttle network to **Slow 3G** in DevTools. 4. Click **Publish Article** 3–4 times rapidly.	**Expected:** Submit button disables immediately on first click; only one POST /api/articles call fires. **Actual:** Multiple concurrent POST /api/articles requests fire, creating duplicate articles or 422 errors.	**High**	Frontend form handler does not bind an isSubmitting state lock to the button's disabled attribute while the async API promise is pending.

2	**JWT Session Token Stored in Plaintext localStorage (XSS Vulnerability)**	1. Log in with a valid account. 2. Open DevTools (F12) -> **Application** -> **Local Storage** -> https://demo.realworld.io. 3. Inspect the jwtToken key.	**Expected:** Auth tokens should be stored in HttpOnly, Secure, SameSite=Strict cookies. **Actual:** Raw Bearer JWT is stored in plaintext inside localStorage, accessible to any script.	**Critical**	AI-scaffolded auth service persists tokens via window.localStorage.setItem('jwtToken', token) instead of server-set HttpOnly cookies.
3	**Silent Failure / Infinite Loading State When Feed API Times Out or Returns 5xx**	1. Open home page (/#/). 2. In DevTools **Network** tab, block GET /api/articles (or set network to Offline). 3. Refresh the page.	**Expected:** An error banner with a "Retry" button is shown to the user. **Actual:** UI stays stuck on "Loading articles..." forever with an unhandled promise rejection in console.	**High**	Missing .catch() error boundary state in the feed component; loading flag is only cleared inside the success callback.

4	**No Whitespace Validation on Article Creation & Comment Submission**	1. Log in and open **New Article** or any article's comment box. 2. Type only spaces (" ") into the fields. 3. Click **Publish Article** / **Post Comment**.	**Expected:** Client validation trims whitespace (.trim()) and blocks blank submissions. **Actual:** Form submits whitespace strings directly to the API, creating empty comment cards or broken slugs.	**Medium**	Form validation checks raw string length without calling .trim() before dispatching payload, and lacks HTML5 required pattern rules.
5	**Unauthenticated Click on "Favorite Article" Heart Button Fails Without User Prompt**	1. Log out of the app. 2. On Global Feed, click the green heart (**Favorite**) button on any article. 3. Observe UI and Network tab.	**Expected:** Shows a clear toast/modal prompting the user to sign in. **Actual:** Fires an unauthenticated 401 API call or abruptly changes routes without explaining why the	**Medium**	The FavoriteButton handler dispatches the API mutation before checking authentication state or displaying contextual feedback.

			action failed.		
6	**Accessibility (a11y) & UX Defect: Pagination Renders 50+ Unwindowed Links Without ARIA**	1. Open Global Feed on demo.realworld.io. 2. Scroll to bottom pagination bar. 3. Inspect .pagination DOM elements and navigate with Tab.	**Expected:** Windowed pagination (1 2 3 ... 50 Next) with aria-label and aria-current="page". **Actual:** Renders dozens of raw numbered links wrapping across multiple lines without screen-reader landmarks.	**Low**	Naive Array.from({ length: totalPages }) loop rendered directly in template without pagination windowing or WAI-ARIA attributes.

2. Root-Cause Analysis (Issue #1: Missing Idempotency & Debouncing on Article Creation)

When a user clicks the "Publish Article" button on demo.realworld.io, the frontend form submission handler dispatches an asynchronous POST /api/articles HTTP request to the backend without locking the UI state. Because the submit button's disabled attribute is not bound to an in-flight request boolean (or the state update occurs asynchronously after event propagation), rapid clicks or high-latency connections allow the user to trigger multiple identical network calls before the browser receives the first 200/201 response and redirects. On the backend, the API attempts to generate a URL slug from the article title for each simultaneous request, resulting in either duplicate database entries with appended random suffixes or unhandled 422 Unprocessable Entity race-condition errors. This is a classic issue in AI-generated ("vibe-coded") web apps, where happy-path CRUD wiring is generated without defensive concurrency controls. To fix this on the frontend, we should bind an

isSubmitting boolean state (or TanStack Mutation's isPending flag) that synchronously sets disabled={isSubmitting} on the submit button and displays a loading indicator the instant onSubmit fires. In the catch block, isSubmitting is reset to false only if the request fails, preventing duplicate clicks prior to route navigation. On the backend API layer, we should enforce idempotency by requiring an Idempotency-Key header (UUIDv4 generated per form mount) cached in Redis for 60 seconds so duplicate requests return the original cached response instead of re-running the database insert.